Discussion 1

Introduction to Socket Programming

&

A1 Introduction

Outline

- Logistics
- A1 Intro
- Introduction to socket programming

Logistics

Who am I?

- I'm Efe!
- Finished my undergrad last year, finishing up SUGS this year
- International student from Turkey 🦃, living in Ann Arbor since high school
- IA for 281 for 4 semesters, second semester as a GSI for 489
- Moving to Miami for job next year! 🌴
- Ask me about
  - IA / GSI
  - International student things: (OPT/CPT)
  - Job stuff

Figure 1: Me

Logistics

- Course website on GitHub
- Complete VM setup ASAP!
  - Ubuntu instances on the VM will also be used for A4!
- A1 is out! Due **Wednesday, Jan. 29**
  - A1 focuses on creating a tool similar to iPerf, which measures the speed of a network connection.
  - All assignments are submitted through the Autograder
  - A1 is individual, while the rest can be completed in groups of 2-3 people.

Assignement 1
Introduction

VM Setup

- For assignment 1, you will be using a virtual machine (VM)
- There is a VM setup guide in the same repo as the project
- VM setup is a time commitment of anywhere from 30 minutes to an hour!
  - Get started as soon as possible!

Assignment 1 — iPerfer

- In assignment 1, you will be implementing a program that measures the bandwidth and latency of sending data over TCP in a network
- You will use socket programming to achieve this task
  - iPerfer server: waits for a client to connect, receives data, and sends `ACKs`[1]
  - iPerfer client: sends data to a iPerfer client
- Both the server and client measure the observed rate of data and latency; display statistics at the end

[1] These are application level `ACKs`, not TCP `ACKs` that are used within the protocol

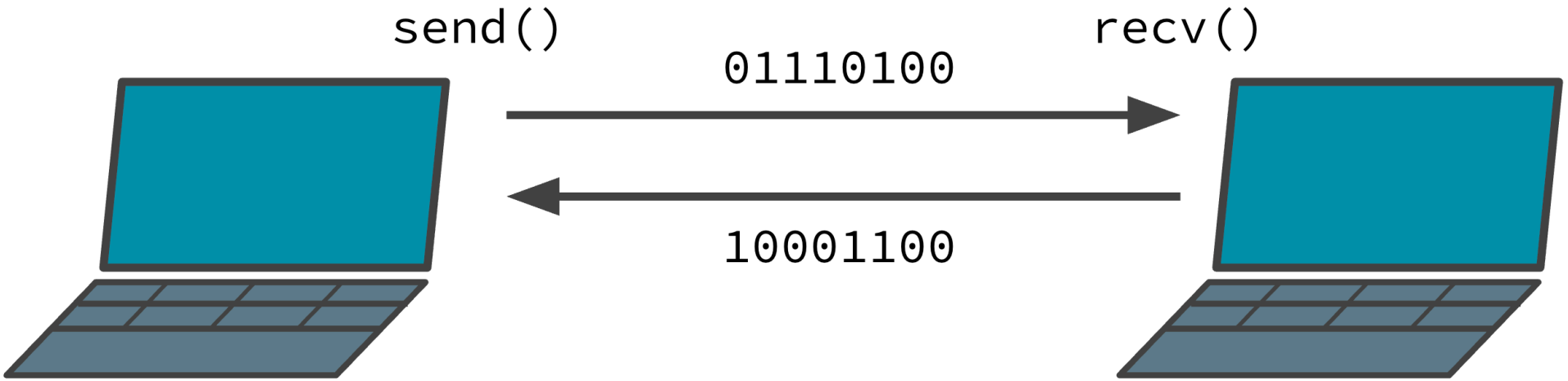
Assignment 1 — iPerfer

- Important links:
  - Spec: https://github.com/eecs489staff/a1-sockets-mininet
  - Introduction to Modern CMake: https://cliutils.gitlab.io/modern-cmake/chapters/basics.html
  - Beej's Guide to Network Programming: https://beej.us/guide/bgnet/

Intro to Socket Programming

What is a Socket?

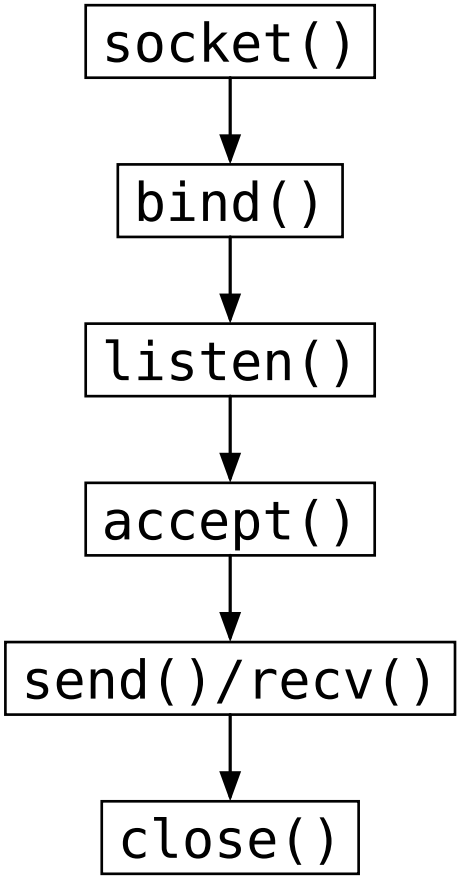
- A socket is a two-way communication channel between two processes or machines
- Sockets abstract away many parts of communicating over a network
  - Provides a programming interface (API) to exchange data
  - Hides details of interacting with network card



Socket API

```
1   int socket(domain, type, protocol);
2
3   ssize_t send(sockfd, buffer, size, flags);
4   ssize_t recv(sockfd, buffer, size, flags);
5
6   int bind   (sockfd, addr);
7   int listen (sockfd, backlog);
8   int accept (sockfd, 0, 0);
9   int connect(sockfd, addr);
10  int close  (sockfd);
```

Server Flow



Create a socket

Bind the socket to a port

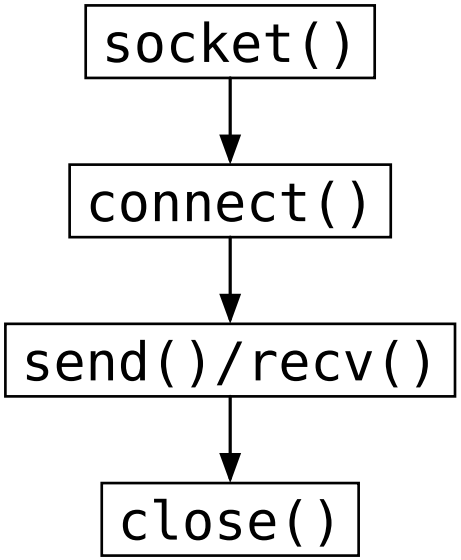
Begin listening for incoming connections

Accept a connection request

Send or receive stream of bytes

Close the socket

Client Flow



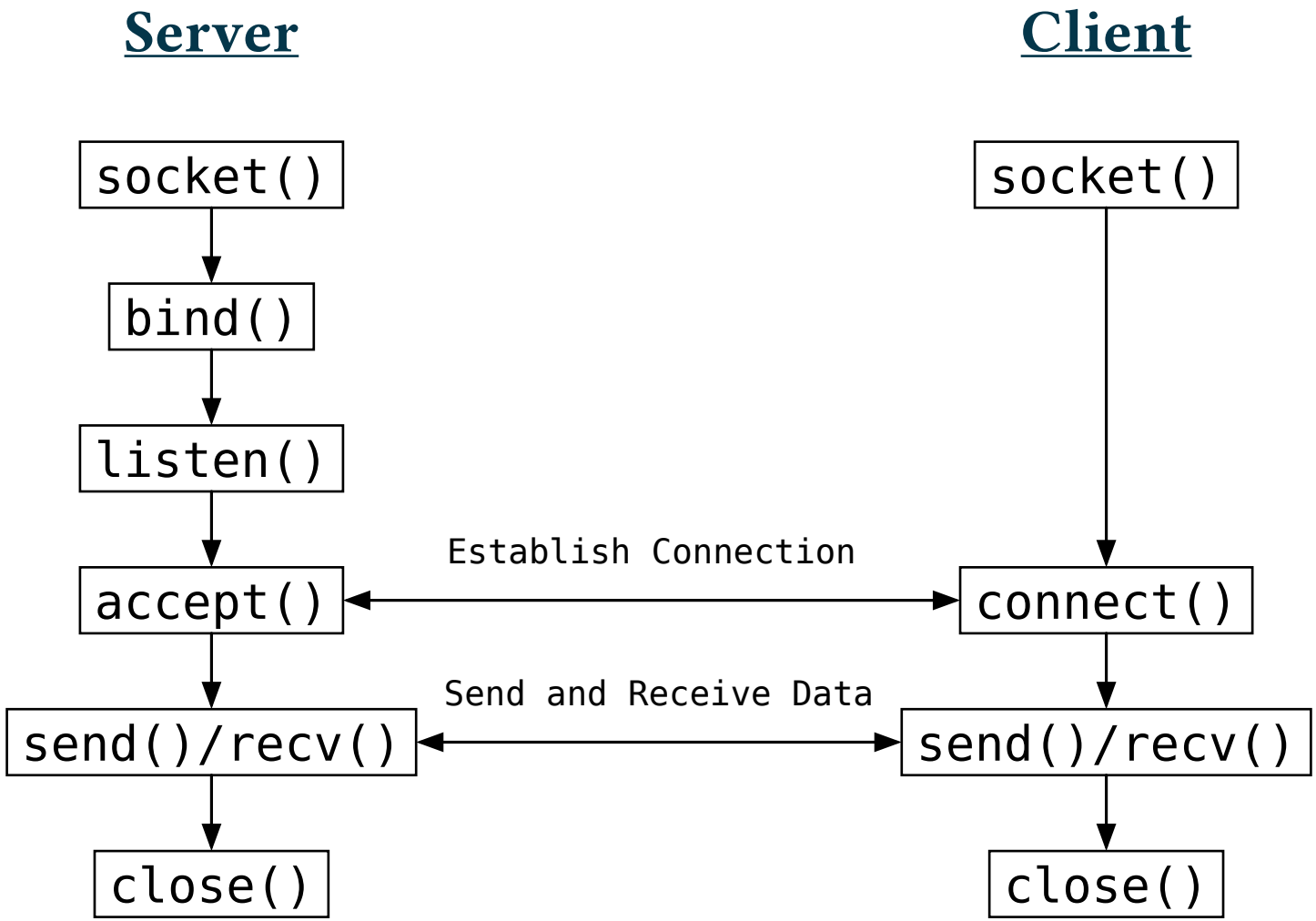
Create a socket

Connect to a server

Send or receive stream of bytes

Close the socket

Server and Client Interaction



socket()

```
socket(int domain, int type, int protocol);
```

- **domain:** Specifies the communication domain; `AF_INET` is for IPv4 protocol
- **type:** Specifies the communication semantics; for a two-way bytestream communication socket, we use `SOCK_STREAM`
- **protocol:** Specifies the protocol; this will be `IPPROTO_TCP` for our purposes

Note: These symbols are defined in `sys/socket.h`

```
1 // ...
2 int sockfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
3 ssize_t bytes_sent = send(sockfd, my_vec.data(), my_vec.size(), 0);
```

bind()

```
int bind(int sockfd, const struct sockaddr_in *addr,
  socklen_t addrlen);
```

Binds the socket to a specific port, among other things.

- **sockfd:** The socket file descriptor from `socket()`
- **addr:** A `sockaddr` struct that specifies some parameters, including which port to be used (via `addr.sin_port`)
  - If this is set to 0, the OS chooses a port for you
  - The `sockaddr` type is a generalization of `sockaddr_in`, which is the internet type
- **addrlen:** `sizeof(sockaddr)`

Returns `0` on success, `-1` on error.

Aside: What's a Port?

- A port is a logical channel (number) used alongside an IP address to route data to specific services or applications

The well-known ports (also known as *system ports*) are those numbered from 0 through 1023. The requirements for new assignments in this range are stricter than for other registrations.[2]

Notable well-known port numbers

Number	Assignment
20	File Transfer Protocol (FTP) Data Transfer
21	File Transfer Protocol (FTP) Command Control
22	Secure Shell (SSH) Secure Login
23	Telnet remote login service, unencrypted text messages
25	Simple Mail Transfer Protocol (SMTP) email delivery
53	Domain Name System (DNS) service
67, 68	Dynamic Host Configuration Protocol (DHCP)
80	Hypertext Transfer Protocol (HTTP) used in the World Wide Web
110	Post Office Protocol (POP3)
119	Network News Transfer Protocol (NNTP)
123	Network Time Protocol (NTP)
143	Internet Message Access Protocol (IMAP) Management of digital mail
161	Simple Network Management Protocol (SNMP)
194	Internet Relay Chat (IRC)
443	HTTP Secure (HTTPS) HTTP over TLS/SSL
546, 547	DHCPv6 IPv6 version of DHCP

The registered ports are those from 1024 through 49151. IANA maintains the official list of well-known and registered ranges.[3]

The dynamic or private ports are those from 49152 through 65535. One common use for this range is for ephemeral ports.

Aside: sockaddr_in


```
1  int make_server_sockaddr(struct sockaddr_in *addr,
2      int port) {
3    // Step (1): specify socket family.
4    // This is an internet socket.
5    addr->sin_family = AF_INET;
6    // Step (2): specify socket address(hostname).
7    // The socket will be a server, so it will only be
     listening.
8    // Let the OS map it to the correct address.
9    addr->sin_addr.s_addr = INADDR_ANY;
10   // Step (3): Set the port value.
11   // If port is 0, the OS will choose the port for us.
12   // Use htons to convert from local byte order to
     network byte
13   addr->sin_port = htons(port);
14   return 0;
15 }
```

```
1  int make_client_sockaddr(struct sockaddr_in addr,
2      const char *hostname, int port) {
3    // Step (1): specify socket family.
4    // This is an internet socket.
5    addr->sin_family = AF_INET;
6    // Step (2): specify socket address (hostname).
7    // The socket will be a client, so call this unix
     helper function
8    // to convert a hostname string to a usable 'hostent'
     struct.
9    struct hostent *host = gethostbyname(hostname);
10   if (host == NULL) {
11     printf(stderr, "%s: unknown host\n", hostname);
12     return -1;
13   }
14   memcpy(&addr->sin_addr, host->h_addr, host->h_length);
15   // Step (3): Set the port value.
16   // Use htons to convert from local byte order to
     network byte order.
17   addr->sin_port = htons(port);
18   return 0;
19 }
```

Aside: Errors and `errno`

- Suppose that a function like `bind()` or `send()` returns `-1`, indicating an error
  - How do we know what went wrong?
- Calls such as these will set a global[1] `errno` variable indicating what went wrong
  - `errno` can be found in `<errno.h>`
  - `errno` is just a number, but functions like `strerror` output a meaningful code

```
1 int success = bind(sockfd, &bind_addr, sizeof(bind_addr));
2 if (success == -1) {
3   spdlog::error("Error while binding to socket: {}", strerror(errno));
4 }
```

[1] Actually thread-local, not global

listen()

```
int listen(int sockfd, int backlog);
```

- **sockfd:** The socket to mark as being used to accept incoming connections
- **backlog:** The maximum length the queue of connections waiting to be accepted can grow to

connect()

```
int connect(int sockfd, const struct sockaddr_in *addr,
  socklen_t addrlen);
```

- **sockfd:** The socket file descriptor from `socket()`
- **addr:** A `sockaddr_in` struct specifying the server's IP address and port
- **addrlen:** The size of the `sockaddr_in` struct

Attempts to establish a connection to the server. Returns `0` on success, `-1` on error.

send()

```
ssize_t send(int sockfd, const void *buf, size_t len, int
flags);
```

- **sockfd:** The socket file descriptor used to send data
- **buf:** A pointer to the data to send
- **len:** The number of bytes to send
- **flags:** Additional options for sending; typically `0` for standard behavior

Returns the number of bytes sent, or `-1` on error.

send() — Example


```cpp
std::vector<char> message{'H', 'e', 'l', 'l', 'o', '!', '\0'};
size_t total_bytes_sent = 0;

while (total_bytes_sent < message.size()) {
    ssize_t bytes_sent = send(sockfd,
      message.data() + total_bytes_sent,
      message.size() - total_bytes_sent, 0);

    if (bytes_sent == -1) {
        spdlog::error("Send error: {}", strerror(errno));
        break; // Exit the loop on error
    }
    total_bytes_sent += bytes_sent; // Update the count of sent bytes
}
```

recv()

```
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

- **sockfd:** The socket file descriptor used to receive data
- **buf:** A pointer to the buffer where received data will be stored
- **len:** The maximum number of bytes to read
- **flags:** Options for receiving; commonly `0` for standard behavior
  - Useful: `MSG_WAITALL` will wait to receive `len` bytes before returning

Returns the number of bytes received, `0` for end-of-stream (connection closed), or `-1` on error.

```
1  char buffer[1024];
2  ssize_t bytes_received = recv(sockfd, buffer, sizeof(buffer), 0);
```

Common Error — `sizeof`

- `sizeof` is a common source of errors in 489
- What does the following code print?

```
1    std::vector<char> my_vec;
2    for (size_t i = 0; i < 100; ++i) {
3      my_vec.push_back('a');
4    }
5
6    spdlog::info("Sizeof my_vec: {}", sizeof(my_vec))
```

- A) `100`
- B) `4`
- C) `8`
- D) `Can't be determined by the code alone`

Common Error — `sizeof`

- `sizeof` is a common source of errors in 489
- What does the following code print?

```
1    std::vector<char> my_vec;
2    for (size_t i = 0; i < 100; ++i) {
3      my_vec.push_back('a');
4    }
5
6    spdlog::info("Sizeof my_vec: {}", sizeof(my_vec))
```

- A) `100`
- B) `4`
- C) `8`
- **D) `Can't be determined by the code alone`**

Common Error — `sizeof`

- `sizeof` returns the size of a type
- For example, consider

```
1 struct MyStruct {
2   int a;
3   int b;
4 }
```

  - `sizeof(MyStruct) == 8`, since it contains two ints
- A `std::vector` contains a pointer to some data, plus some bookkeeping variables for keeping track of its size
  - This means that `sizeof(std::vector)` is equal to the size of the bookkeeping variables + the size of a pointer (4 bytes on 32-bit systems, 8 bytes on 64-bit systems).

close()

```
int close(int sockfd);
```

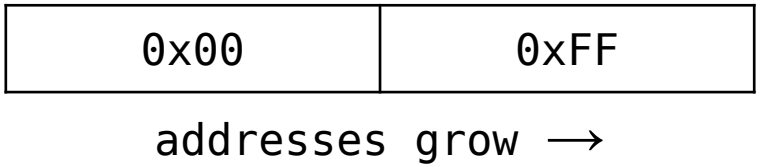
- **sockfd:** The socket file descriptor to close

Closes the socket and releases associated resources. Returns `0` on success, `-1` on error.

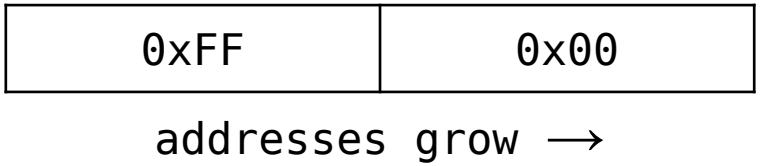
```
1 if (close(sockfd) == -1) {
2   spdlog::error("Close error: {}", strerror(errno));
3 } else {
4   spdlog::info("Socket successfully closed");
5 }
```

Note - Byte Order

- When storing numbers, different computers order the most-significant bytes in different ways
- For example, consider the two-byte integer `65280` (`0xFF00`).
  - In a *little-endian* system, the least significant byte is stored at the lowest address



  - In a *big-endian* system, the most significant byte is stored at the lowest address



Note - Byte Order

- What happens if a little-endian system sends a number to a big-endian system?
  - `send/recv` just send bytes in the order they're in memory
- To avoid this issue, we always convert numbers to *network-order* (big-endian) before sending them over networks

Function	Meaning
`ntohl`	**n**etwork **t**o **h**ost **l**ong
`ntohs`	**n**etwork **t**o **h**ost **s**hort
`htonl`	**h**ost **t**o **n**etwork **l**ong
`htons`	**h**ost **t**o **n**etwork **s**hort

Resources & Advice

- Beej's Guide to Network Programming
  - Example code: https://github.com/wlfuh/bgreeves.sockets.starter/tree/master
  - Feel free to copy-paste portions of this code for your Assignment 1 as necessary!
- Use existing documentation (i.e. man pages) – these are generally very complete!
  - Nice UI wrapper: https://man7.org/linux/man-pages/
- StackOverflow and ChatGPT can be very helpful for socket programming
  - Many of these things are formulaic; others likely have encountered similar problems
  - Understanding code instead of pure copy-paste can help prevent further questions down the line

Helpful Headers to #include


```
1 #include <arpa/inet.h> // ntohl() et al.
2 #include <netdb.h> // gethostbyname(), struct hostent
3 #include <netinet/in.h> // struct sockaddr_in
4 #include <sys/socket.h> // getsockname()
5 #include <chrono> // std::chrono::*
```